

▼ **AIM :** Use sequence data types and their associated operations in Python

- a. Strings
- b. Lists
- c. Arrays
- d. Tuples
- e. Dictionary
- f. Sets
- g. Range

Theory :

1. What are sequence data types in Python? Name them and explain indexing and slicing.

Sequence data types in Python are data structures that store multiple values in an ordered manner, where each element is accessible using an index. The main sequence data types include strings, lists, tuples, arrays, and range. In all these types, indexing is used to access elements using their position, starting from 0. For example, in a list or string, the first element is at index 0, the second at index 1, and so on. Negative indexing is also supported, where -1 refers to the last element. Slicing is used to access a subset of elements using a range of indices, written as `sequence[start:end:step]`. It allows extraction of parts of sequences efficiently. This concept works similarly across strings, lists, tuples, and arrays, making data manipulation flexible and powerful.

2. What is the difference between a list and a tuple in Python? Why prefer tuple?

A list and a tuple are both sequence data types in Python, but they differ mainly in mutability. A list is mutable, which means its elements can be changed, added, or removed after creation. On the other hand, a tuple is immutable, meaning its elements cannot be modified once it is created. Lists are defined using square brackets `[]`, while tuples use parentheses `()`. Tuples are generally faster and consume less memory compared to lists. In real-world applications, tuples are preferred when the data should remain constant, such as storing coordinates, fixed configurations, or database records. Using tuples ensures data integrity and prevents accidental modification.

3. How are Python arrays different from lists? When to use arrays?

Python arrays, created using the `array` module, are different from lists in terms of data type consistency. Lists can store elements of different data types, whereas arrays can only store elements of the same data type, making them more memory-efficient. Arrays are also faster for numerical operations compared to lists. Lists are more flexible and widely used, but arrays are preferred when working with large numerical datasets or when memory optimization is required. Arrays are commonly used in scientific computing, mathematical operations, and when handling homogeneous data.

4. How do dictionaries work internally in Python? Why are they faster?

Dictionaries in Python store data in the form of key-value pairs and are implemented using a hash table internally. Each key is passed through a hash function, which generates a unique index where the value is stored. This allows direct access to data using the key instead of searching sequentially. Because of this hashing mechanism, dictionary lookups are very fast, typically taking constant time $O(1)$. In contrast, lists and tuples require linear search ($O(n)$) to find an element. This is why dictionaries are highly efficient for searching, retrieving, and storing data where quick access is required.

5. What is the difference between sets and lists? How do sets handle duplicates?

Sets and lists are both used to store collections of elements, but they differ in several ways. A list is an ordered collection that allows duplicate values, whereas a set is an unordered collection that does not allow duplicates. When duplicate values are added to a set, they are automatically removed, ensuring all elements are unique. Sets also support mathematical operations such as union, intersection, and difference, which are not available in lists. Sets are useful in practical scenarios where unique values are required, such as removing duplicates from data, checking membership efficiently, and performing set operations.

String

```
# String example
s = "Hello Python"

print("Original String:", s)

# Indexing
print("First character:", s[0])
print("Last character:", s[-1])

# Slicing
print("Slicing (1:3):", s[1:3])
print("Slicing (5:):", s[5:])
print("Reverse string:", s[::-1])
```

```
Original String: Hello Python
First character: H
Last character: n
Slicing (1:3): el
Slicing (5:): Python
Reverse string: nohtyP olleH
```

List

```
# List example
lst = [5, 15, 25, 35, 45]

print("\nOriginal List:", lst)

# Indexing
print("First element:", lst[0])

# Slicing
print("Slice (1:3):", lst[1:3])

# Operations
lst.append(50)
lst.remove(25)

print("Modified List:", lst)
```

```
Original List: [5, 15, 25, 35, 45]
First element: 5
Slice (1:3): [15, 25]
Modified List: [5, 15, 35, 45, 50]
```

Array

```
import array

# Create array
arr = array.array('i', [5, 10, 20, 30, 40])

print("\nOriginal Array:", arr)

# Indexing
print("First element:", arr[0])

# Slicing
print("Slice (1:4):", arr[1:4])

# Append
arr.append(8)

print("Modified Array:", arr)
```

```
Original Array: array('i', [5, 10, 20, 30, 40])
First element: 5
Slice (1:4): array('i', [10, 20, 30])
Modified Array: array('i', [5, 10, 20, 30, 40, 8])
```

Tuple

```
# Tuple example
tup = (45, 55, 65, 75)

print("\nOriginal Tuple:", tup)

# Indexing
print("First element:", tup[0])

# Slicing
print("Slice (1:3):", tup[1:3])
```

```
Original Tuple: (45, 55, 65, 75)
First element: 45
Slice (1:3): (55, 65)
```

Dictionary

```
# Dictionary example
d = {'name': 'Vaishnavi', 'age': 20, 'course': 'Btech'}

print("\nDictionary:", d)

# Access values
print("Name:", d['name'])

# Add new key-value
d['city'] = 'Chhatrapati Sambhajinagar'

# Update value
d['age'] = 21

print("Updated Dictionary:", d)
```

```
Dictionary: {'name': 'Vaishnavi', 'age': 20, 'course': 'Btech'}
Name: Vaishnavi
Updated Dictionary: {'name': 'Vaishnavi', 'age': 21, 'course': 'Btech', 'city': 'Chhatrapati Sambhajinagar'}
```

Sets

```
# Set example
s = {10, 20, 30, 40, 50}

print("\nSet:", s)

# Add element
s.add(6)

# Set operations
set2 = {45, 55, 65, 75}

print("Union:", s.union(set2))
print("Intersection:", s.intersection(set2))
```

```
Set: {50, 20, 40, 10, 30}
Union: {4, 5, 6, 7, 40, 10, 50, 20, 30}
Intersection: {6}
```

Range

```
# Range example
r = range(1, 10)

print("\nRange values:")
for i in r:
    print(i, end=" ")
```

```
Range values:
1 2 3 4 5 6 7 8 9
```

Conclusion

This practical focused on exploring different sequence data types in Python such as strings, lists, arrays, tuples, dictionaries, sets, and ranges. It involved using indexing and slicing techniques to retrieve elements, along with performing various operations on each type. Through this, a clearer understanding of how these data structures work and their real-world uses was developed.

